

21. Bölüm

Bellek Yönetimi ve Kaynak Denetimi

Yüksek performanslı kurumsal uygulamalarda doğru yazmak yetmez; *verimli* yazmak da gerekir. .NET'in çöp toplayıcısı (GC) pek çok bellek işini otomatik üstlense de, yanlış yazılmış kod gereksiz tahsisler, uzun çöp toplama duraklamaları ve yönetilmeyen kaynak sızıntılarıyla sistemin performansını ciddi biçimde düşürebilir.

Bu bölümde .NET bellek modelini, sıfır tahsisli (**zero-allocation**) dilim yapılarını, dizi havuzlarını ve yönetilmeyen kaynakların güvenli serbest bırakılmasını öğreneceksiniz. Bölümün sonunda aşağıdaki soruları rahatlıkla yanıtlatabileceksiniz:

- Yığın (**stack**) ve öbek (**heap**) bellek arasındaki fark nedir?
- `Span<T>` neden öbek tahsisi yapmadan dilim oluşturabilir?
- `Memory<T>` ile `Span<T>` ne zaman birbirinin yerine geçemez?
- `ArrayPool<T>` kullanarak çöp toplayıcı (GC) baskısı nasıl azaltılır?
- `IDisposable` deseni doğru nasıl uygulanır?
- Eş zamansız kaynak temizleme için `IAsyncDisposable` ne zaman tercih edilir?
- Bellek sızıntısı hangi araçlarla tespit edilir?

21.1 .NET'te Bellek Modeli

§21.1.1 Yığın ve Öbek Bellek

.NET çalışma zamanı belleği iki ana bölgede yönetir: **yığın** (**stack**) ve **öbek** (**heap**). Bu iki bölgenin davranışını anlamak, performanslı kod yazmanın temelidir.

Yığın bellek (**stack**), iş parçacığına özgüdür ve son giren ilk çıkar (LIFO) mantığıyla çalışır. Yöntem çağrıldığında yerel değişkenler ve parametreler yığına itilir (**push**); yöntem sonlandığında otomatik olarak çıkarılır (**pop**). Tahsis ve serbest bırakma işlemi bir işlemci komutuyla gerçekleşir çöp toplayıcı devreye girmez. Ancak yığın boyutu sınırlıdır (varsayılan olarak 1 MB).

Öbek bellek (**heap**), uygulamanın genel bellek havuzudur. **new** anahtar kelimesiyle oluşturulan tüm nesneler buraya tahsis edilir. Çöp toplayıcı bu bölgeyi izler, erişilemeyen nesneleri tespit eder ve belleklerini geri alır. Tahsis esnektir ancak çöp toplayıcı duraklamaları performansı etkileyebilir.

Özellik	Yığın	Öbek
Tahsis hızı	Çok hızlı (tek komut)	Nispeten yavaş
Serbest bırakma	Otomatik (kapsam sonu)	Çöp toplayıcı tarafından
Boyut sınırı	Küçük (~1 MB)	Büyük (RAM ile sınırlı)
Kapsam	İş parçacığına özgü	Uygulama genelinde paylaşılr
Depolanan tipler	Değer tipleri, referanslar	Referans tipi nesneler
Çöp toplayıcı baskısı	Yok	Var

Tablo 21.1: Yığın ve Öbek Bellek Karşılaştırması

§21.1.2 Değer Tipleri ve Referans Tipleri

```
// Değer tipi - yığında yaşar
int x = 42;
double pi = 3.14;
DateTime bugün = DateTime.Now;

// Referans tipi - öbekte yaşar, yığında yalnızca referans tutulur
string ad = "İlhan";
int[] dizi = new int[100];
List<string> liste = new List<string>();

// Yapı (struct) - değer tipidir, yığında yaşar
// Sınıf (class) - referans tipidir, öbekte yaşar
```

```
struct Nokta { public int X; public int Y; } // yığın
class NesnNokta { public int X; public int Y; } // öbek
```

§21.1.3 .NET Çöp Toplayıcı Kuşakları

.NET çöp toplayıcı (GC), nesneleri üç kuşakta (**generation**) yönetir. Bu yapı, kısa ömürlü nesnelerin hızla temizlenmesini, uzun ömürlü nesnelerin ise seyrek kontrol edilmesini sağlar.

Kuşak	İçerik	Çöp Toplama Sıklığı	Boyut
Kuşak 0	Yeni tahsis edilen nesneler	Çok sık	~256 KB
Kuşak 1	Kuşak 0'dan hayatta kalanlar	Orta	~2 MB
Kuşak 2	Uzun ömürlü nesneler	Seyrek	Sınırsız
Büyük Nesne Yığını	85 KB üzeri nesneler	Seyrek	Sınırsız

Tablo 21.2: Çöp Toplama Kuşakları ve Özellikleri

```
// Hangi kuşakta olduğunu öğrenme
object nesne = new object();
Console.WriteLine($"Kuşak: {GC.GetGeneration(nesne)}"); // Çıktı: 0

GC.Collect(); // Zorla GC (üretim kodunda kullanmayın!)
Console.WriteLine($"Kuşak: {GC.GetGeneration(nesne)}"); // Çıktı: 1

// Bellek baskısını izleme
long kullanılanBellek = GC.GetTotalMemory(forceFullCollection: false);
Console.WriteLine($"Kullanılan bellek: {kullanılanBellek / 1024} KB");
```

§21.1.4 stackalloc ile Yığın Tahsisi

Küçük, kısa ömürlü diziler için **stackalloc** ile yığında dizi oluşturulabilir. Öbek tahsisi yapılmaz, çöp toplama devreye girmez.

```
using System;

class StackallocÖrneği
{
    static void Main()
    {
        // Yığında 256 bayt ayır - öbek tahsisi yok
        Span<byte> tampon = stackalloc byte[256];

        for (int i = 0; i < tampon.Length; i++)
            tampon[i] = (byte)i;

        Console.WriteLine($"İlk eleman: {tampon[0]}");
        Console.WriteLine($"Son eleman: {tampon[255]}");
        // Yöntem sonunda otomatik serbest bırakılır
    }
}

/*Program Çıktısı:
İlk eleman: 0
Son eleman: 255
*/
```

21.2 Yığın Tabanlı Bellek Dilimi

Yığın Tabanlı Bellek Dilimi **Span<T>**, C#7.3+ ile tanıtılan ve **sıfır tahsisli** (**zero-allocation**) bellek dilimleri oluşturmayı sağlayan bir yapıdır (**struct**). Mevcut bir bellek bölgesinin dizi gibi, yığın (**stack**) tahsisi veya yönetilmeyen bellek üzerine bir pencere açar; yeni bellek tahsis etmez.

Temel kural: **Span<T>** bir **ref struct**'tır. Bu nedenle yalnızca yığında yaşayabilir; öbekte (**heap**) depolanamaz. Sınıf alanı olamaz, **async** yöntemlerde ve **lambda** ifadelerinde kullanılamaz.

§21.2.1 Dizi Üzerinde Dilim Oluşturma

```
using System;

class SpanTemelÖrneği
{
    static void Main()
    {
        int[] dizi = { 10, 20, 30, 40, 50, 60, 70, 80 };

        // Tüm dizi – kopyalama yok
        Span<int> tümü = dizi;

        // 2. indeksten başlayan 4 elemanlık dilim – kopyalama yok
        Span<int> dilim = dizi.AsSpan(2, 4); // {30, 40, 50, 60}

        Console.WriteLine("Dilim elemanları:");
        foreach (int eleman in dilim)
            Console.Write($"{eleman} ");
        Console.WriteLine();

        // Dilim üzerinde değişiklik orijinal diziyi etkiler
        dilim[0] = 999;
        Console.WriteLine($"Orijinal dizi[2]: {dizi[2]}"); // 999
    }
}

/*Program Çıktısı:
Dilim elemanları:
30 40 50 60
Orijinal dizi[2]: 999
*/
```

§21.2.2 Metin İşlemede Span<T> ve Sıfır Tahsis

`string` işlemleri genellikle yeni `string` nesneleri oluşturur ve öbek baskısı yaratır. `ReadOnlySpan<char>` bu sorunu çözer.

```
using System;

class MetinDilimÖrneği
{
    // KÖTÜ: Her Substring çağrısı yeni string nesnesi tahsis eder
    static string KlasikParçala(string kaynak, char ayraç)
    {
        int indeks = kaynak.IndexOf(ayraç);
        return indeks >= 0
            ? kaynak.Substring(0, indeks) // yeni string tahsisi!
            : kaynak;
    }

    // İYİ: Sıfır tahsis – mevcut metin üzerinde pencere açar
    static ReadOnlySpan<char> SpanParçala(ReadOnlySpan<char> kaynak, char ayraç)
    {
        int indeks = kaynak.IndexOf(ayraç);
        return indeks >= 0
            ? kaynak.Slice(0, indeks) // tahsis yok!
            : kaynak;
    }

    static void Main()
    {
        string veri = "İlhan ÖZKAN;Ankara;Türkiye";

        // Klasik yol – üç yeni string nesnesi
```

```

        string[] parçalar = veri.Split(';');
        Console.WriteLine($"Ad: {parçalar[0]}");

        // Span yolu – sıfır ek tahsis
        ReadOnlySpan<char> span = veri.AsSpan();
        ReadOnlySpan<char> ad = SpanParçala(span, ';');
        Console.WriteLine($"Ad (Span): {ad.ToString()}");
    }
}
/*Program Çıktısı:
Ad: İlhan ÖZKAN
Ad (Span): İlhan ÖZKAN
*/

```

§21.2.3 stackalloc ile Span<T> Birlikte Kullanımı

```

using System;

class YığınTahsisÖrneği
{
    // Küçük tamponu yığında oluştur, GC baskısı sıfır
    static int[] SıralıKopyala(ReadOnlySpan<int> kaynak)
    {
        // Küçük ara tampon – yığında, öbekte değil
        Span<int> geçici = stackalloc int[kaynak.Length];
        kaynak.CopyTo(geçici);

        // Yalnızca sonuç öbeğe gider
        geçici.Sort();
        return geçici.ToArray();
    }

    static void Main()
    {
        int[] veri = { 5, 2, 8, 1, 9, 3 };
        int[] sıralı = SıralıKopyala(veri);

        Console.Write("Sıralı: ");
        foreach (int e in sıralı)
            Console.Write($"{e} ");
        Console.WriteLine();
    }
}
/*Program Çıktısı:
Sıralı: 1 2 3 5 8 9
*/

```

§21.2.4 Span<T> Kısıtlamaları

```

// DERLEME HATASI – Span sınıf alanı olamaz
class YanlışKullanım
{
    private Span<int> _alan; // HATA: ref struct sınıf alanı olamaz
}

// DERLEME HATASI – Span async yöntemde kullanılamaz
async Task YanlışAsync()
{
    Span<int> span = new int[10]; // HATA: async metodda ref struct
    await Task.Delay(100);
}

// DOĞRU – Bu kısıtlamalar için Memory<T> kullanılır
class DoğruKullanım

```

```

{
    private Memory<int> _alan; // Memory sınıf alanı olabilir

    async Task DoğruAsync()
    {
        Memory<int> bellek = new int[10]; // async içinde çalışır
        await Task.Delay(100);
    }
}

```

21.3 Öbek Uyumlu Bellek Dilimi

Öbek Uyumlu Bellek Dilimi `Memory<T>`, `Span<T>`'nin öbek uyumlu kardeşidir. `Span<T>`'nin tüm sıfır-tahsis avantajlarını sunar; ancak `ref struct` kısıtlaması yoktur. Sınıf alanı olabilir, `async` yöntemlerde kullanılabilir ve `lambda` ifadelerine geçilebilir.

§21.3.1 `Span<T>` ve `Memory<T>` Karşılaştırması

Özellik	<code>Span<T></code>	<code>Memory<T></code>
Yapı türü	ref struct	struct
Sınıf alanı olabilir mi?	Hayır	Evet
<code>async</code> içinde çalışır mı?	Hayır	Evet
Yığın tahsisi (<code>stackalloc</code>)	Evet	Hayır
<code>Span<T></code> 'e dönüşüm	—	<code>.Span</code>
Performans	Biraz daha hızlı	Çok yakın

Tablo 21.3: `Span<T>` ve `Memory<T>` Karşılaştırması

§21.3.2 Temel Kullanım

```

using System;
using System.Threading.Tasks;

class MemoryÖrneği
{
    // Memory sınıf alanı olabilir
    private Memory<byte> _tampon;

    public MemoryÖrneği(int boyut)
    {
        _tampon = new byte[boyut];
    }

    // async yöntemde kullanılabilir
    public async Task DoldurAsync(byte değer)
    {
        await Task.Delay(10); // simüle edilmiş async iş

        // .Span ile anlık Span<T>'ye dönüştür
        Span<byte> span = _tampon.Span;
        span.Fill(değer);
    }

    public void YazdırİlkN(int n)
    {
        ReadOnlySpan<byte> görünüm = _tampon.Span.Slice(0, n);
        foreach (byte b in görünüm)
            Console.Write($"{b} ");
        Console.WriteLine();
    }
}

class Program

```

```

{
    static async Task Main()
    {
        MemoryÖrneği örnek = new(16);
        await örnek.DoldurAsync(0xAB);
        örnek.YazdırİlkN(8);
    }
}
/*Program Çıktısı:
171 171 171 171 171 171 171 171
*/

```

§21.3.3 Memory<T> ile Büyük Veri İşleme

Büyük bir dosyanın parçalar hâlinde (chunked) okunması, her parça için ayrı dizi tahsis etmeden yapılabilir.

```

using System;
using System.IO;
using System.Threading.Tasks;

class DosyaİşlemcisiMemory
{
    // Tek büyük tampon – döngü boyunca yeniden kullanılır
    private readonly Memory<byte> _tampon;

    public DosyaİşlemcisiMemory(int tamponBoyutu = 4096)
    {
        _tampon = new byte[tamponBoyutu];
    }

    public async Task İşleAsync(string dosyaYolu)
    {
        await using FileStream akış = File.OpenRead(dosyaYolu);

        int okunanBayt;
        long toplamBayt = 0;

        // Her döngüde aynı tampon yeniden kullanılır – tahsis yok
        while ((okunanBayt = await akış.ReadAsync(_tampon)) > 0)
        {
            // Yalnızca okunan kısmı işle
            ReadOnlyMemory<byte> parça = _tampon.Slice(0, okunanBayt);
            toplamBayt += okunanBayt;

            // Parça üzerinde işlem yap
            İşleParça(parça.Span);
        }

        Console.WriteLine($"Toplam işlenen: {toplamBayt} bayt");
    }

    private void İşleParça(ReadOnlySpan<byte> parça)
    {
        // Örnek: baytları say
        Console.WriteLine($"Parça boyutu: {parça.Length} bayt");
    }
}

```

21.4 Dizi Havuzu ile Tahsis Optimizasyonu

Her istek için yeni dizi oluşturmak çöp toplayıcıya (GC) ciddi baskı uygular. `ArrayPool<T>`, dizileri havuzda (pool) tutarak yeniden kullanımı sağlar. Tahsis yerine *ödünç alma* (rent) ve *iade* (return) vardır.

§21.4.1 Temel Kullanım

```
using System;
using System Buffers;

class ArrayPoolTemelÖrneği
{
    static void Main()
    {
        // Paylaşılan (shared) havuzdan ödünç al
        int[] dizi = ArrayPool<int>.Shared.Rent(1000);

        try
        {
            // Dikkat: Rent istenen boyuttan BÜYÜK dizi verebilir
            Console.WriteLine($"İstenen: 1000, Alınan: {dizi.Length}");

            // Diziyi kullan
            for (int i = 0; i < 1000; i++)
                dizi[i] = i * 2;

            int toplam = 0;
            for (int i = 0; i < 1000; i++)
                toplam += dizi[i];

            Console.WriteLine($"Toplam: {toplam}");
        }
        finally
        {
            // MUTLAKA iade et – aksi hâlde havuz tükenir
            // clearArray: true – güvenlik için sıfırla (hassas veri varsa)
            ArrayPool<int>.Shared.Return(dizi, clearArray: false);
        }
    }
}

/*Program Çıktısı:
İstenen: 1000, Alınan: 1024
Toplam: 999000
*/
```

§21.4.2 IDisposable Benzeri Sarmalayıcı ile Güvenli Kullanım

`finally` bloğunu unutmak riski taşır. Sarmalayıcı yapı bu riski ortadan kaldırır.

```
using System;
using System Buffers;

// IDisposable ile ArrayPool dizisini otomatik iade et
readonly struct KiralanmışDizi<T> : IDisposable
{
    private readonly T[] _dizi;
    private readonly int _boyut;

    public KiralanmışDizi(int boyut)
    {
        _dizi = ArrayPool<T>.Shared.Rent(boyut);
        _boyut = boyut;
    }

    // Yalnızca istenen kadar göster
    public Span<T> Span => _dizi.AsSpan(0, _boyut);

    public void Dispose()
    {

```

```

        ArrayPool<T>.Shared.Return(_dizi);
    }
}

class Program
{
    static void Veriİşle(int boyut)
    {
        // using bloğu – kapsam sonunda otomatik iade
        using KiralanmışDizi<double> kira = new(boyut);

        Span<double> veri = kira.Span;
        for (int i = 0; i < veri.Length; i++)
            veri[i] = Math.Sqrt(i);

        Console.WriteLine($"{veri.Length - 1} = {veri[^1]:F4}");
    }

    static void Main()
    {
        Veriİşle(10000);
        // Yöntem sonunda dizi havuza otomatik iade edildi
    }
}
/*Program Çıktısı:
9999 = 99.9950
*/

```

§21.4.3 Özel Havuz Oluşturma

Varsayılan paylaşılan havuz çoğu senaryo için yeterlidir. Ancak özel gereksinimler için özel havuz oluşturulabilir.

```

using System;
using System.Buffers;

class ÖzelHavuzÖrneği
{
    // Özel havuz: maksimum dizi boyutu ve havuz kapasitesi ayarlanır
    private static readonly ArrayPool<byte> _özelHavuz =
        ArrayPool<byte>.Create(
            maxArrayLength: 1024 * 1024, // Maks 1 MB dizi
            maxArraysPerBucket: 50);    // Her boyut kümesinde 50 dizi

    public static void ÖzelHavuzKullan()
    {
        byte[] tampon = _özelHavuz.Rent(512 * 1024); // 512 KB ödünç al
        try
        {
            tampon.AsSpan().Fill(0xFF);
            Console.WriteLine($"Özel havuzdan alınan boyut: {tampon.Length}");
        }
        finally
        {
            _özelHavuz.Return(tampon);
        }
    }
}

```

§21.4.4 Performans Karşılaştırması

Aşağıdaki iki yöntem işlevsel olarak aynıdır; ancak bellek tahsis davranışları çok farklıdır.

```

using System;
using System.Buffers;

```



```

class PerformansKarşılaştırma
{
    const int İterasyon = 100_000;
    const int DiziBoyutu = 1024;

    // KÖTÜ: Her çağrıda yeni dizi – 100.000 × 1024 bayt öbek tahsis
    static void KlasikYöntem()
    {
        for (int i = 0; i < İterasyon; i++)
        {
            byte[] tampon = new byte[DiziBoyutu]; // her seferinde tahsis
            tampon.AsSpan().Fill(0xAA);
        }
    }

    // İYİ: Havuzdan ödünç al – yeniden kullanım, GC baskısı minimum
    static void HavuzluYöntem()
    {
        for (int i = 0; i < İterasyon; i++)
        {
            byte[] tampon = ArrayPool<byte>.Shared.Rent(DiziBoyutu);
            try
            {
                tampon.AsSpan(0, DiziBoyutu).Fill(0xAA);
            }
            finally
            {
                ArrayPool<byte>.Shared.Return(tampon);
            }
        }
    }
}

```

Ölçüm	Klasik new[]	ArrayPool
Toplam tahsis	~100 MB	~1 KB
Çöp toplayıcıdaki koleksiyon sayısı	Yüksek	Neredeyse sıfır
Ortalama süre	Daha yavaş	Belirgin hızlı

Tablo 21.4: Klasik Dizi ve ArrayPool Karşılaştırması (100.000 iterasyon, 1 KB dizi)

21.5 Yönetilmeyen Kaynakların Serbest Bırakılması

.NET çöp toplayıcı (GC), yalnızca *yönetilen* (**managed**) belleği temizler. Dosya tutamaçları (**file handle**), ağ soketleri, veri tabanı bağlantıları ve grafik kaynakları gibi *yönetilmeyen* (**unmanaged**) kaynaklar çöp toplayıcı tarafından otomatik serbest bırakılmaz. Bu kaynakları zamanında temizlemek programcının sorumluluğundadır. .NET bu amaç için **IDisposable** arayüzünü sunar.

§21.5.1 Temel IDisposable Uygulaması

```

using System;

class DosyaYazıcısı : IDisposable
{
    private System.IO.StreamWriter? _yazıcı;
    private bool _temizlendi = false; // Çift temizlemeyi önle

    public DosyaYazıcısı(string dosyaYolu)
    {
        _yazıcı = new System.IO.StreamWriter(dosyaYolu);
        Console.WriteLine($"Dosya açıldı: {dosyaYolu}");
    }
}

```

```

public void Yaz(string metin)
{
    if (!_temizlendi)
        throw new ObjectDisposedException(nameof(DosyaYazıcısı));
    _yazıcı!.WriteLine(metin);
}

public void Dispose()
{
    if (!_temizlendi)
    {
        _yazıcı?.Flush();
        _yazıcı?.Close();
        _yazıcı = null;
        _temizlendi = true;
        Console.WriteLine("Dosya kapatıldı ve kaynaklar serbest bırakıldı.");
    }
    // GC'ye bu nesneyi sonlandırıcı kuyruğuna alma demek
    GC.SuppressFinalize(this);
}
}

class Program
{
    static void Main()
    {
        // using bloğu – kapsam sonunda Dispose otomatik çağrılır
        using (DosyaYazıcısı yazıcı = new("çıkıttı.txt"))
        {
            yazıcı.Yaz("Birinci satır");
            yazıcı.Yaz("İkinci satır");
        } // Burada Dispose() otomatik çağrılır

        Console.WriteLine("Program devam ediyor...");
    }
}
/*Program Çıktısı:
Dosya açıldı: çıkıttı.txt
Dosya kapatıldı ve kaynaklar serbest bırakıldı.
Program devam ediyor...
*/

```

§21.5.2 Hem Yönetilen Hem Yönetilmeyen Kaynaklar

Bir sınıf hem yönetilen hem yönetilmeyen kaynak tutuyorsa **kullan-at deseni** (**disposable pattern**) uygulanmalıdır. Bu desen sonlandırıcı (**finalizer**) ile birlikte çalışır.

```

using System;
using System.Runtime.InteropServices;

class KarmaşıkKaynak : IDisposable
{
    // Yönetilmeyen kaynak (örnek: yerel kütüphane tutamacı)
    private IntPtr _yönetilmeyenKaynak;

    // Yönetilen kaynak
    private System.IO.MemoryStream? _akış;

    private bool _temizlendi = false;

    public KarmaşıkKaynak()
    {
        // Yönetilmeyen bellek tahsis et (örnek amaçlı)
    }
}

```

```

        _yönetilmeyenKaynak = Marshal.AllocHGlobal(1024);
        _akış = new System.IO.MemoryStream();
        Console.WriteLine("Kaynaklar tahsis edildi.");
    }

    // IDisposable.Dispose - kullanıcı tarafından çağrılır
    public void Dispose()
    {
        Temizle(yönetilenKaynaklarıTemizle: true);
        GC.SuppressFinalize(this); // Sonlandırıcıyı iptal et
    }

    // Sonlandırıcı - GC tarafından çağrılır (yalnızca yönetilmeyen kaynaklar için)
    ~KarmaşıkKaynak()
    {
        Temizle(yönetilenKaynaklarıTemizle: false);
    }

    // Asıl temizleme mantığı
    protected virtual void Temizle(bool yönetilenKaynaklarıTemizle)
    {
        if (!_temizlendi)
        {
            if (yönetilenKaynaklarıTemizle)
            {
                // Yönetilen kaynakları temizle
                _akış?.Dispose();
                _akış = null;
                Console.WriteLine("Yönetilen kaynaklar temizlendi.");
            }

            // Yönetilmeyen kaynakları her durumda temizle
            if (_yönetilmeyenKaynak != IntPtr.Zero)
            {
                Marshal.FreeHGlobal(_yönetilmeyenKaynak);
                _yönetilmeyenKaynak = IntPtr.Zero;
                Console.WriteLine("Yönetilmeyen bellek serbest bırakıldı.");
            }

            _temizlendi = true;
        }
    }
}

```

§21.5.3 Modern using Bildirimi

C#8.0+ ile gelen `using` bildirimi, parantez ve blok yazmayı ortadan kaldırır. Kaynak, bulunduğu kapsamın sonunda otomatik temizlenir.

```

using System;
using System.IO;

class ModernUsingÖrneği
{
    static void Main()
    {
        // Klasik using bloğu
        using (StreamReader klasik = new("veri.txt"))
        {
            string? satır = klasik.ReadLine();
            Console.WriteLine(satır);
        }
    }
}

```

```

// Modern using bildirimi (C# 8.0+) – daha az girintileme
using StreamReader modern = new("veri.txt");
string? satır2 = modern.ReadLine();
Console.WriteLine(satır2);
// Yöntem sonu veya kapsam sonu geldiğinde Dispose çağrılır

// Birden fazla kaynak
using StreamReader okuyucu = new("kaynak.txt");
using StreamWriter yazıcı = new("hedef.txt");
string? içerik;
while ((içerik = okuyucu.ReadLine()) != null)
    yazıcı.WriteLine(içerik.ToUpper());
}
}

```

§21.5.4 Hata Durumunda Kaynak Güvenliği

using bloğu içinde istisna fırlatılsa bile Dispose her zaman çağrılır.

```

using System;
using System.IO;

class HataGüvenliğiÖrneği
{
    static void Main()
    {
        try
        {
            using StreamWriter yazıcı = new("çıktı.txt");
            yazıcı.WriteLine("Birinci satır");

            throw new InvalidOperationException("Bir hata oluştu!"); // Hata fırlat

            yazıcı.WriteLine("Bu satır yazılmaz."); // Erişilmez kod
        }
        catch (InvalidOperationException hata)
        {
            Console.WriteLine($"Hata yakalandı: {hata.Message}");
        }
        // İstisna fırlatılmış olsa da yazıcı.Dispose() çağrıldı
        Console.WriteLine("Dosya güvenli kapatıldı.");
    }
}

/*Program Çıktısı:
Hata yakalandı: Bir hata oluştu!
Dosya güvenli kapatıldı.
*/

```

21.6 Eş Zamansız Kaynak Temizleme

Bazı kaynakların temizlenmesi eş zamansız işlem gerektirir: ağ bağlantılarının kibarca kapatılması, tampon verilerin diske aktarılması, uzak oturumların sonlandırılması. C#8.0 ile gelen `IAsyncDisposable` ve `await using` bu senaryolar için tasarlanmıştır.

§21.6.1 Temel Uygulama

```

using System;
using System.Threading.Tasks;

class EşZamansızBağlantı : IAsyncDisposable
{
    private bool _açık = false;

    public async Task BağlanAsync(string sunucu)
    {
        // ...
    }
}

```

```

        await Task.Delay(100); // Bağlantı kurma simülasyonu
        _açık = true;
        Console.WriteLine($"Bağlantı kuruldu: {sunucu}");
    }

    public async Task VeriGönderAsync(string veri)
    {
        if (!_açık)
            throw new InvalidOperationException("Bağlantı kapalı.");

        await Task.Delay(50); // Ağ gecikmesi
        Console.WriteLine($"Gönderildi: {veri}");
    }

    // Eş zamansız temizleme
    public async ValueTask DisposeAsync()
    {
        if (_açık)
        {
            // Bekleyen verileri temizle ve bağlantıyı kapat
            await Task.Delay(80); // Kapatma handshake
            _açık = false;
            Console.WriteLine("Bağlantı kibarca kapatıldı.");
        }
        GC.SuppressFinalize(this);
    }
}

class Program
{
    static async Task Main()
    {
        // await using – kapsam sonunda DisposeAsync otomatik çağrılır
        await using EşZamansızBağlantı bağlantı = new();

        await bağlantı.BağlanAsync("sunucu.example.com");
        await bağlantı.VerigönderAsync("Merhaba Sunucu");
        await bağlantı.VerigönderAsync("İkinci paket");
    } // Burada await DisposeAsync() otomatik çağrılır
}

/*Program Çıktısı:
Bağlantı kuruldu: sunucu.example.com
Gönderildi: Merhaba Sunucu
Gönderildi: İkinci paket
Bağlantı kibarca kapatıldı.
*/

```

§21.6.2 Hem IDisposable Hem IAsyncDisposable Uygulama

Bir sınıf her iki arayüzü de uygulayabilir. Eş zamansız bağlamda `await using`, eş zamanlı bağlamda `using` çalışır.

```

using System;
using System.IO;
using System.Threading.Tasks;

class EsnekKaynak : IDisposable, IAsyncDisposable
{
    private FileStream? _akış;

    public EsnekKaynak(string yol)
    {
        _akış = new FileStream(yol,

```

```

        FileMode.OpenOrCreate, FileAccess.Write,
        FileShare.None, bufferSize: 4096,
        useAsync: true); // Eş zamansız G/Ç etkin
    }

    public async Task YazAsync(byte[] veri)
    {
        if (_akış is null)
            throw new ObjectDisposedException(nameof(EsnekKaynak));
        await _akış.WriteAsync(veri);
    }

    // Eş zamansız temizleme (tercih edilir)
    public async ValueTask DisposeAsync()
    {
        if (_akış is not null)
        {
            await _akış.FlushAsync(); // Tamponu diske yaz
            await _akış.DisposeAsync();
            _akış = null;
        }
        GC.SuppressFinalize(this);
    }

    // Eş zamanlı temizleme (yedek)
    public void Dispose()
    {
        _akış?.Flush();
        _akış?.Dispose();
        _akış = null;
        GC.SuppressFinalize(this);
    }
}

class Program
{
    static async Task Main()
    {
        // Eş zamansız bağlamda
        await using (EsnekKaynak kaynakAsync = new("çıkkt1.bin"))
        {
            await kaynakAsync.YazAsync(new byte[] { 1, 2, 3, 4 });
        }

        // Eş zamanlı bağlamda
        using (EsnekKaynak kaynakSync = new("çıkkt12.bin"))
        {
            // Eş zamanlı kullanım
        }
    }
}

```

§21.6.3 IDisposable ve IAsyncDisposable Karşılaştırması

Özellik	IDisposable	IAsyncDisposable
Anahtar kelime	using	await using
Temizleme yöntemi	Dispose()	DisposeAsync()
Dönüş tipi	void	ValueTask
Eş zamansız giriş-çıkış (IO) destekler mi?	Hayır	Evet
Ne zaman tercih edilir?	Eş zamanlı kaynaklar	Ağ, dosya, veri tabanı bağlantıları

Tablo 21.5: IDisposable ve IAsyncDisposable Karşılaştırması

21.7 Bellek Sızıntısı Tespiti ve Profilleme

.NET'in çöp toplaması çoğu bellek sorununu otomatik çözer; ancak bazı yaygın hatalar sızıntıya (**memory leak**) yol açar. Bu bölümde sızıntı kalıpları ve tespit yöntemleri incelenmektedir.

§21.7.1 Yaygın Bellek Sızıntısı Kalıpları

Olay İşleyicisi Sızıntısı

En yaygın sızıntı kaynağı abonelikten çıkılmayan olay işleyicileridir. İşleyiciyi tutan nesne canlı kaldığı sürece abone olan nesne de çöp toplayıcı (GC) tarafından toplanamaz.

```
using System;

class YayıncıSınıf
{
    public event EventHandler? Olay;

    public void OlayıTetikle() => Olay?.Invoke(this, EventArgs.Empty);
}

class AboneSınıf
{
    private readonly YayıncıSınıf _yayıncı;

    public AboneSınıf(YayıncıSınıf yayıncı)
    {
        _yayıncı = yayıncı;
        // SORUNLU: Bu abone nesne Dispose edilse bile
        // yayıncı bu nesneyi referans tutmaya devam eder
        _yayıncı.Olay += Olayİşleyici;
    }

    private void Olayİşleyici(object? gönderen, EventArgs e)
    {
        Console.WriteLine("Olay alındı.");
    }

    // ÇÖZÜM: IDisposable uygula ve abonelikten çık
    public void Dispose()
    {
        _yayıncı.Olay -= Olayİşleyici; // Abonelikten çık!
    }
}
```

Statik Koleksiyonda Nesne Biriktirme

```
using System;
using System.Collections.Generic;

class SızıntıÖrneği
{
    // SORUNLU: Statik liste hiç temizlenmez
    private static readonly List<byte[]> _önbellek = new();

    public static void VeriEkle(int boyut)
    {
        // Her çağrıda listeye eklenir, hiç çıkarılmaz
        _önbellek.Add(new byte[boyut]);
    }

    // ÇÖZÜM: Zaman sınırlı önbellek veya WeakReference kullan
    private static readonly List<WeakReference<byte[]>> _zayıfÖnbellek = new();

    public static void ZayıfVeriEkle(byte[] veri)
```

```

{
    // GC gerektiğinde bu nesneleri toplayabilir
    _zayıfÖnbellek.Add(new WeakReference<byte[]>(veri));
}
}

```

Nesneleri Temizlememe

IDisposable ile nesneleri temizleme yöntemine aşağıda örnek verilmiştir;

```

using System;
using System.Net.Http;

class HttpSızıntısı
{
    // SORUNLU: Her çağrıda yeni HttpClient – soket tüketimi
    public static async System.Threading.Tasks.Task<string> KötüYöntemAsync()
    {
        HttpClient istemci = new(); // using yok!
        return await istemci.GetStringAsync("https://example.com");
        // istemci.Dispose() hiç çağrılmaz – soket açık kalır
    }

    // ÇÖZÜM 1: using ile temizle
    public static async System.Threading.Tasks.Task<string> İyiYöntemAsync()
    {
        using HttpClient istemci = new();
        return await istemci.GetStringAsync("https://example.com");
    }

    // ÇÖZÜM 2 (tercih): IHttpClientFactory ile ömür yönetimi
    // DI konteyneri HttpClient ömrünü otomatik yönetir
}

```

§21.7.2 Bellek Kullanımını İzleme

```

using System;

class Bellekİzleme
{
    static void Main()
    {
        // Anlık bellek kullanımı
        long başlangıç = GC.GetTotalMemory(forceFullCollection: false);
        Console.WriteLine($"Başlangıç: {başlangıç / 1024:N0} KB");

        // Büyük veri tahsis et
        byte[][] veriler = new byte[100][];
        for (int i = 0; i < 100; i++)
            veriler[i] = new byte[1024 * 1024]; // 100 × 1 MB

        long sonra = GC.GetTotalMemory(forceFullCollection: false);
        Console.WriteLine($"Tahsis sonrası: {sonra / 1024:N0} KB");
        Console.WriteLine($"Fark: {(sonra - başlangıç) / 1024:N0} KB");

        // Referansı bırak
        veriler = null!;
        long temizlik = GC.GetTotalMemory(forceFullCollection: true);
        Console.WriteLine($"GC sonrası: {temizlik / 1024:N0} KB");

        // Kuşak bilgisi
        Console.WriteLine($"Kuşak 0 sayısı: {GC.CollectionCount(0)}");
        Console.WriteLine($"Kuşak 1 sayısı: {GC.CollectionCount(1)}");
        Console.WriteLine($"Kuşak 2 sayısı: {GC.CollectionCount(2)}");
    }
}

```



```

    }
}

```

§21.7.3 Profilleme Araçları

Bellek sorunlarını tespit etmek için çeşitli araçlar kullanılabilir.

Araç	Tür	Kullanım Amacı
dotnet-counters	CLI	Anlık çöp toplama ve bellek metrikleri
dotnet-dump	CLI	Bellek anlık görüntüsü (snapshot)
dotnet-trace	CLI	Olay izleme ve çöp toplama profili
Visual Studio Diagnostic Tools	IDE	Anlık görüntü karşılaştırma
JetBrains dotMemory	Ticari	Sızıntı analizi, nesne ağacı
BenchmarkDotNet	Açık kaynak	Tahsis miktarı ölçümü

Tablo 21.6: .NET Bellek Profilleme Araçları

```

// dotnet-counters ile izleme (komut satırı)
// dotnet-counters monitor --process-id <PID> System.Runtime

// BenchmarkDotNet ile tahsis ölçümü
using BenchmarkDotNet.Attributes;
using BenchmarkDotNet.Running;

[MemoryDiagnoser] // Tahsis miktarını ölç
public class TahsisBenchmark
{
    [Benchmark]
    public string KlasikBirleştir()
    {
        string sonuç = "";
        for (int i = 0; i < 100; i++)
            sonuç += i.ToString(); // Her iterasyonda yeni string
        return sonuç;
    }

    [Benchmark]
    public string SpanBirleştir()
    {
        return string.Create(292, 0, (span, _) =>
        {
            int konum = 0;
            for (int i = 0; i < 100; i++)
            {
                i.TryFormat(span.Slice(konum), out int yazılan);
                konum += yazılan;
            }
        });
    }
}

// BenchmarkRunner.Run<TahsisBenchmark>();

```

§21.7.4 Bellek Yönetimi En İyi Uygulamaları

21.8 Düşün ve Çöz

- ◇ **Düşün:** `Span<T>` neden `async` yöntemlerde kullanılamaz? Derleyicinin `async` yöntemi durum makinesine dönüştürme süreciyle nasıl ilişkilidir? Bu kısıtlamayı aşmak için hangi yapıyı kullanırsınız?
- ◇ **Düşün:** Aşağıdaki kodda bellek sızıntısı var mı? Varsa nereden kaynaklanmaktadır ve nasıl düzeltilir?

```

class Uygulama
{
    private static List<Action> _eylemler = new();
}

```

Uygulama	Neden?
<code>IDisposable</code> nesneleri <code>using</code> ile kullan	Kaynak sızıntısını önler
Eş zamansız bağlamda <code>await using</code> kullan	Kapatma işlemi bloke etmez
Olay aboneliğinden <code>Dispose</code> 'da çık	Nesne sızıntısını önler
Büyük diziler için <code>ArrayPool</code> kullan	Çöp toplama baskısını azaltır
Dilim işlemleri için <code>Span<T></code> kullan	Kopyalamayı ortadan kaldırır
<code>async</code> bağlamda <code>Memory<T></code> kullan	<code>Span<T></code> <code>async</code> içinde çalışmaz
<code>stackalloc</code> boyutunu küçük tut	Yığın taşmasını (<code>StackOverflow</code>) önler
Sonlandırıcı (<code>finalizer</code>) yazmaktan kaçın	Çöp toplayıcı nesneyi iki kez işler, yavaşlatır

Tablo 21.7: Bellek Yönetimi En İyi Uygulamaları

```
static void KaydetEylem(string ad)
{
    string yerelAd = ad; // yerel değişken
    _eylemler.Add(() => Console.WriteLine(yerelAd));
}
```

- ◇ **Çöz:** Bir CSV (**Comma Seperated Values**) dosyasını satır satır okuyan ve her satırdaki alanları virgüle göre ayıran bir yöntem yazınız. `string.Split` yerine `ReadOnlySpan<char>` ve `Slice` kullanarak öbek tahsisini sıfıra indirin. `BenchmarkDotNet` veya `GC.GetTotalMemory` ile iki versiyonun tahsis miktarını karşılaştırın.
- ◇ **Çöz:** Bir görüntü işleme uygulaması saniyede binlerce küçük bayt dizisi kullanmaktadır. `ArrayPool<byte>` kullanarak dizi havuzunu oluşturun. Havuzlu ve havuzsuz versiyonu çalıştırarak `GC.CollectionCount` değerlerini karşılaştırın.
- ◇ **Düşün:** Tam **Dispose deseni** ne zaman gereklidir? Yalnızca yönetilen kaynaklar tutan bir sınıf için sonlandırıcı (`finalizer`) yazmak neden gereksiz ve zararlıdır?
- ◇ **Çöz:** Bir veri tabanı bağlantısı sınıfı tasarlayınız. Sınıf hem `IDisposable` hem `IAsyncDisposable` uygulansın. `Dispose` çağrıldığında bağlantıyı eş zamanlı, `DisposeAsync` çağrıldığında eş zamansız kapatsın. Çift temizlemeyi `_temizlendi` bayrağıyla önleyin.
- ◇ **Düşün:** `WeakReference<T>` ile normal referans arasındaki fark nedir? Büyük resim önbelleği (**image cache**) tasarlarlarken `WeakReference<T>` kullanmak ne gibi avantajlar sağlar? Ne gibi riskler doğurur?